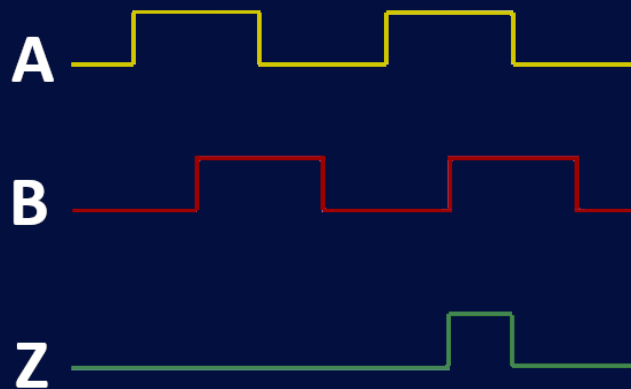
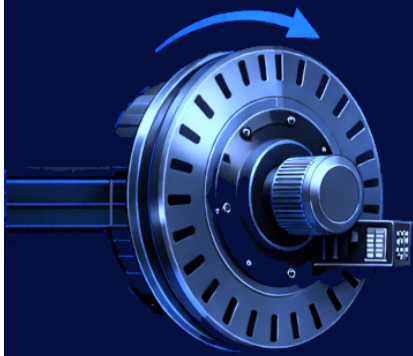


Encoder incrementali: come emularli con un microcontrollore



Leonardo Chieco

Encoder incrementali: come emularli con un microcontrollore

Gli encoder incrementali sono tra i sensori più utilizzati nei sistemi di automazione industriale, nel controllo dei motori elettrici e nei sistemi di movimentazione. Essi permettono di misurare con precisione posizione, velocità e direzione di rotazione di alberi meccanici, fornendo informazioni fondamentali per il funzionamento di azionamenti, robot, macchine automatiche e sistemi CNC. Il principio di funzionamento si basa generalmente sulla generazione di due segnali digitali sfasati di 90° (canali A e B), dai quali è possibile ricavare sia il verso di rotazione sia la risoluzione del movimento; spesso è presente anche un terzo segnale di riferimento (Z o ZERO) che identifica una posizione angolare precisa.

Durante lo sviluppo o il collaudo di sistemi di controllo che utilizzano encoder, può essere molto utile disporre di uno strumento in grado di simulare artificialmente questi segnali senza dover utilizzare un motore o un encoder reale. In questo tutorial vedremo quindi come funziona un encoder incrementale e come sia possibile emularne il comportamento utilizzando un microcontrollore Arduino, generando i segnali A, B e ZERO con temporizzazione precisa tramite i timer hardware del dispositivo. Il risultato sarà un semplice ma efficace emulatore di encoder, utile per test, sviluppo e sperimentazione nei sistemi di controllo del movimento.

Buona lettura!

L'autore

Leonardo Chieco è un ingegnere elettronico con oltre 20 anni di esperienza nella progettazione e sviluppo di software per il controllo dell'automazione (PC/PLC), nella progettazione di schede elettroniche per applicazioni industriali, firmware, robotica e mecatronica.

LinkedIn: <https://www.linkedin.com/in/leonardo-chieco-53550b129/>

Web: <https://leochieco.github.io/>

1. Introduzione

Un encoder è un dispositivo elettromeccanico utilizzato per convertire una grandezza fisica legata al movimento (tipicamente posizione angolare o lineare) in un segnale elettrico interpretabile da un sistema di controllo. In pratica, consente di “tradurre” il moto di un albero o di un carrello in informazioni digitali o analogiche, fondamentali per applicazioni di automazione, robotica e controllo di motori, come ad esempio nei sistemi con motori brushless.

Gli encoder si suddividono principalmente in due grandi categorie: encoder assoluti ed encoder incrementali. I primi forniscono direttamente la posizione univoca dell'elemento in ogni istante, anche dopo uno spegnimento, grazie a una codifica interna (spesso multi-bit); i secondi, invece, generano una sequenza di impulsi proporzionale allo spostamento e richiedono un riferimento iniziale per determinare la posizione.

Un'ulteriore classificazione riguarda la tecnologia di rilevamento: gli encoder possono essere ottici (basati su dischi forati e fotodiodi), magnetici (che sfruttano sensori Hall o magnetoresistivi) oppure capacitivi.

Tra gli encoder incrementali, quelli in quadratura sono particolarmente diffusi perché, grazie a due segnali sfasati tra loro, permettono di determinare non solo la velocità ma anche la direzione del movimento con elevata affidabilità.

Gli encoder incrementali sono dispositivi largamente utilizzati nei sistemi di automazione industriale, nel controllo dei motori elettrici e nei sistemi di misura della posizione e della velocità. Essi generano tipicamente due segnali digitali sfasati di 90° elettrici (canali A e B) che permettono di determinare sia la velocità di rotazione sia il verso di movimento. Molti encoder forniscono inoltre un terzo segnale di riferimento, spesso indicato come Z o ZERO, che identifica una posizione angolare specifica.

Durante la messa in servizio di sistemi di controllo può essere molto utile disporre di uno strumento capace di simulare il comportamento di un encoder reale. Un emulatore di encoder permette infatti di generare segnali A/B con frequenza controllata e perfettamente stabile, rendendo possibile testare hardware e software anche in assenza del motore o dell'encoder fisico.

In questo tutorial verrà illustrato come realizzare un emulatore di encoder incrementale programmabile utilizzando Arduino. Come vedremo, il sistema proposto è in grado di generare segnali in quadratura (A e B) con frequenza regolabile, oltre a un segnale di riferimento ZERO sincronizzato con il treno di impulsi. La frequenza può essere modificata tramite pulsanti, mentre un display grafico OLED mostra in tempo reale i parametri operativi del dispositivo.

Alla fine della guida sapremo come realizzare un emulatore di encoder completo, capace di generare segnali stabili e regolabili, utile sia come strumento didattico sia come supporto nelle attività di sviluppo e collaudo di sistemi di controllo.

2. Encoder in quadratura

Un encoder in quadratura è un trasduttore rotativo (o lineare) che serve a misurare la posizione angolare, la velocità e la direzione di rotazione di un albero meccanico. È costituito da una parte mobile (rotore) e una parte fissa (statore) che, grazie a un sistema ottico, magnetico o capacitivo, genera segnali digitali periodici proporzionali al movimento.

Il termine “*in quadratura*” si riferisce al fatto che i due segnali principali generati dall’encoder, detti A e B, sono sfasati di 90° elettrici (cioè un quarto di periodo). Questo sfasamento è la chiave che permette di determinare la direzione di rotazione:

- se B anticipa A, la rotazione è in senso antiorario.
- se A anticipa B, la rotazione è, ad esempio, in senso orario;

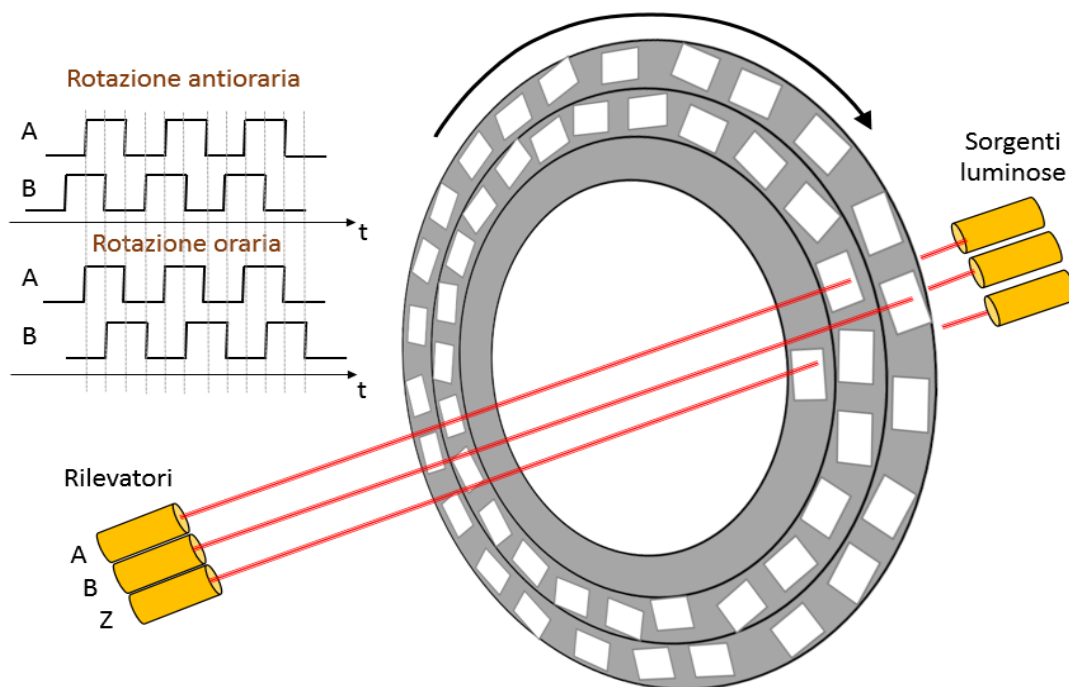


Figura 1: Schema di principio di un encoder ottico in quadratura

In generale, un encoder incrementale standard fornisce tre segnali digitali:

- Segnale A: è un'onda quadra con frequenza proporzionale alla velocità di rotazione.
- Segnale B: è anch'esso un'onda quadra, uguale ad A ma sfasata di 90°. Questo sfasamento consente di distinguere la direzione del moto.
- Segnale Z (o Index, o Zero pulse): è un impulso singolo per ogni giro completo. Serve come riferimento assoluto: quando il segnale Z passa da basso ad alto, l'albero si trova in una posizione angolare nota (zero meccanico).

Il conteggio delle transizioni logiche dei segnali A e B consente di misurare la posizione angolare incrementale, mentre l'impulso Z permette di azzerare il contatore o sincronizzare il riferimento di posizione.

Immaginiamo che l'encoder fornisca N impulsi per giro sul canale A.

Poiché anche il canale B produce N impulsi, ma sfasati di 90°, si ottengono 4 fronti utili per ogni ciclo completo (fronte di salita e discesa di A e B). Questo metodo di conteggio, detto "*x4 decoding*", moltiplica per quattro la risoluzione effettiva. Ad esempio, con un encoder da 1000 impulsi/giro, si ottengono 4000 passi/giro.

La logica di decodifica (hardware o software) osserva la sequenza degli stati dei segnali A,B:

00 → 01 → 11 → 10 → 00 (rotazione oraria)

00 → 10 → 11 → 01 → 00 (rotazione antioraria)

e aggiorna di conseguenza il contatore di posizione.

Nelle applicazioni industriali e negli ambienti con forti disturbi elettromagnetici, i segnali digitali single-ended (TTL o CMOS) possono degradarsi o essere corrotti da interferenze, specialmente quando trasmessi su cavi lunghi.

Per evitare questi problemi, gli encoder professionali usano una trasmissione differenziale bilanciata (es. standard RS-422). Ogni canale logico (A, B, Z) viene quindi trasmesso come coppia differenziale:

A+ / A- B+ / B- Z+ / Z-

La logica ricevente rileva il segnale logico in base alla differenza di potenziale tra i due fili:

- Livello logico "1" $\rightarrow A+ > A-$
- Livello logico "0" $\rightarrow A+ < A-$

In questo modo:

- i disturbi di modo comune (che si accoppiano ugualmente sui due conduttori) vengono cancellati, poiché il ricevitore rileva solo la differenza;
- si ottiene maggiore immunità ai rumori e trasmissione stabile su lunghe distanze (anche decine di metri);
- le correnti di ritorno sullo schermo del cavo sono minime.

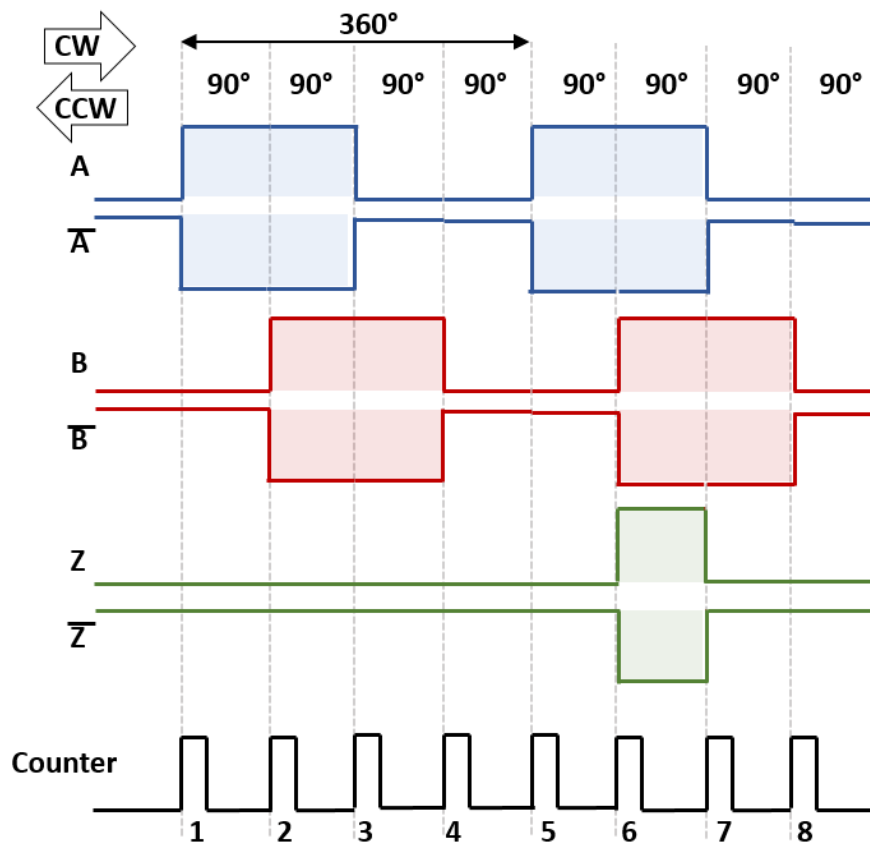


Figura 2: Segnali forniti dall'encoder incrementale in quadratura

Quando si parla di segnali in quadratura, il pensiero va subito agli encoder rotativi o lineari, ma in realtà questa tipologia di segnale è utilizzata anche in molti altri

sensori e trasduttori che non misurano direttamente la posizione, bensì grandezze fisiche differenti come portata, pressione, velocità o intensità luminosa.

Il principio di base resta lo stesso: generare due segnali sfasati di 90° che permettono di dedurre sia la magnitudine del fenomeno, sia la direzione o il verso di variazione.

Esempi di sensori con uscita in quadratura

- Sensori di flusso con palette magnetiche o ottiche
In alcune turbine di misura della portata (per gas o liquidi), due fotodiodi o sensori magnetici vengono posizionati in modo da produrre due impulsi sfasati di 90° . L'elaborazione in quadratura consente di sapere in quale direzione scorre il fluido (utile in sistemi bidirezionali) e di migliorare la risoluzione del conteggio.
- Sensori di velocità su ruote dentate (pick-up magnetici)
Nei tachimetri o nei sistemi di misura della velocità angolare (ad esempio su alberi motore o trasmissioni), due sensori magneto-resistivi o Hall vengono posizionati in modo da leggere il passaggio dei denti con uno sfasamento spaziale equivalente a 90° elettrici. Anche qui, la quadratura permette di distinguere il senso di rotazione e di ottenere conteggi più precisi.

2.2 Modalità di funzionamento

Abbiamo visto che un encoder in quadratura ha due segnali digitali, A e B, sfasati di 90° elettrici. Questi segnali possono essere valutati in tre diversi modi da parte di un sistema di controllo (es. PLC) influenzando direttamente la risoluzione.

Modalità X1

Si conta solo il fronte di salita di un canale (di solito A).

Per un ciclo completo di A (un periodo), si genera un impulso conteggiabile.

È la modalità più semplice ma ha la minor risoluzione.

Modalità X2

Si contano sia il fronte di salita che il fronte di discesa di un canale.

Questo raddoppia il numero di impulsi rispetto a X1, migliorando la risoluzione del conteggio.

Modalità X4

Si contano tutti i fronti di salita e discesa di entrambi i canali A e B.

Ogni ciclo completo genera quindi quattro impulsi conteggiabili, massimizzando la risoluzione possibile dall'encoder.

Questa modalità è la più precisa, ma richiede una elettronica di conteggio leggermente più sofisticata.

Modalità	Fronti Contati	Moltiplicatore di Risoluzione	Caso d'Uso Tipico
X1	Solo fronte di salita del Canale A (o B)	1×	Applicazioni ad alta velocità
X2	Fronti di salita e discesa del Canale A (o B)	2×	Requisiti bilanciati di risoluzione e frequenza
X4	Tutti i fronti di entrambi i canali A e B	4×	Controllo del movimento, posizionamento di precisione

3. Emulatore

Un emulatore di segnali in quadratura è uno strumento estremamente utile per chi si occupa di sviluppo, collaudo o manutenzione di sistemi che ricevono ingressi da encoder o sensori con uscite A/B/Z. Un dispositivo di questo tipo consente infatti di testare schede di controllo, PLC e sistemi di acquisizione simulando diverse condizioni operative, come velocità e direzione di rotazione. In questo modo è possibile verificare il corretto funzionamento degli algoritmi di conteggio, eseguire diagnostica sui sistemi di acquisizione e svolgere attività di manutenzione e troubleshooting direttamente in campo.

Quando si desidera generare segnali in quadratura a frequenze relativamente elevate, la sola gestione software diventa spesso insufficiente, soprattutto nel caso di piattaforme come Arduino. La generazione dei fronti con il corretto sfasamento temporale richiede infatti una temporizzazione molto precisa, che difficilmente può essere garantita esclusivamente tramite codice senza introdurre jitter o limitazioni di frequenza. Per questo motivo è opportuno affiancare al microcontrollore una semplice logica hardware dedicata.

Nel progetto descritto in questo tutorial, Arduino viene utilizzato principalmente come generatore di clock. Il microcontrollore produce un'onda quadra su due pin digitali di uscita, con frequenza regolabile fino a circa 10 kHz, che costituisce la base temporale dell'intero sistema. I due pin vengono utilizzati per selezionare il verso di rotazione simulato. Uno genera il segnale di avanzamento per il conteggio in senso orario, mentre l'altro abilita il conteggio in senso antiorario.

Questi segnali vengono inviati agli ingressi UP e DOWN di un contatore binario, nello specifico il modello SN74193. Il contatore ha il compito di scorrere ciclicamente gli stati binari da 0 a 3, producendo in uscita una sequenza ordinata di combinazioni logiche. I due bit meno significativi del contatore (Q0 e Q1) costituiscono quindi una codifica ciclica a due bit che rappresenta le quattro configurazioni logiche fondamentali associate alle fasi A e B di un encoder incrementale.

L'uscita Q_0 del contatore rappresenta già il segnale B, mentre l'XOR tra Q_0 e Q_1 fornisce il segnale A.

Dunque, con una semplice rete combinatoria riusciamo a mappare opportunamente i segnali Q_0 e Q_1 in modo da ottenere due uscite digitali sfasate di 90° elettrici, ovvero le classiche uscite in quadratura A e B. Il risultato è una coppia di segnali digitali che riproduce fedelmente il comportamento di un encoder incrementale reale, nel quale la direzione del movimento viene determinata dall'ordine temporale con cui si presentano i fronti logici delle due fasi.

Il sistema ottenuto è semplice, economico e molto robusto: la temporizzazione principale è generata da Arduino, mentre la sequenza corretta delle fasi è garantita dalla logica hardware del contatore, permettendo così di ottenere segnali in quadratura stabili anche a frequenze relativamente elevate.

Stato		Uscite	
Q_0	Q_1	$A = Q_0 \oplus Q_1$	$B = Q_1$
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	1

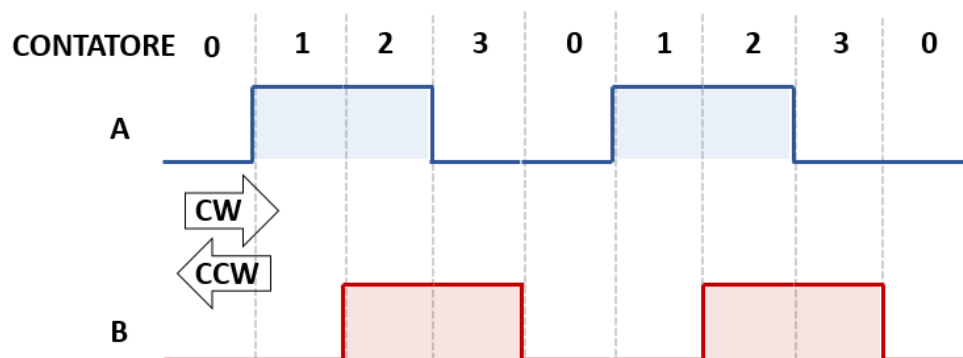


Figura 3: Codifica degli stati del contatore

I due segnali single-ended, A e B, vengono successivamente applicati agli ingressi di un driver differenziale a quattro canali (modello SN75192). Questo integrato converte ciascun segnale TTL in una coppia differenziale bilanciata ($A+/A-$ e $B+/B-$), generando tensioni simmetriche rispetto al riferimento comune.

L'uso della trasmissione differenziale, conforme allo standard RS-422, offre una maggiore immunità ai disturbi elettromagnetici e permette di trasportare i segnali su cavi lunghi senza degrado del segnale o falsi impulsi.

In questo modo, con pochi componenti, economici e facilmente reperibili, si può costruire un emulatore di encoder digitale pienamente compatibile con gli ingressi differenziali di PLC, motion controller o sistemi di misura industriali, utile per test, calibrazioni o validazioni di firmware dedicati al conteggio in quadratura.

Nella figura seguente è mostrato lo schema di principio dell'emulatore.

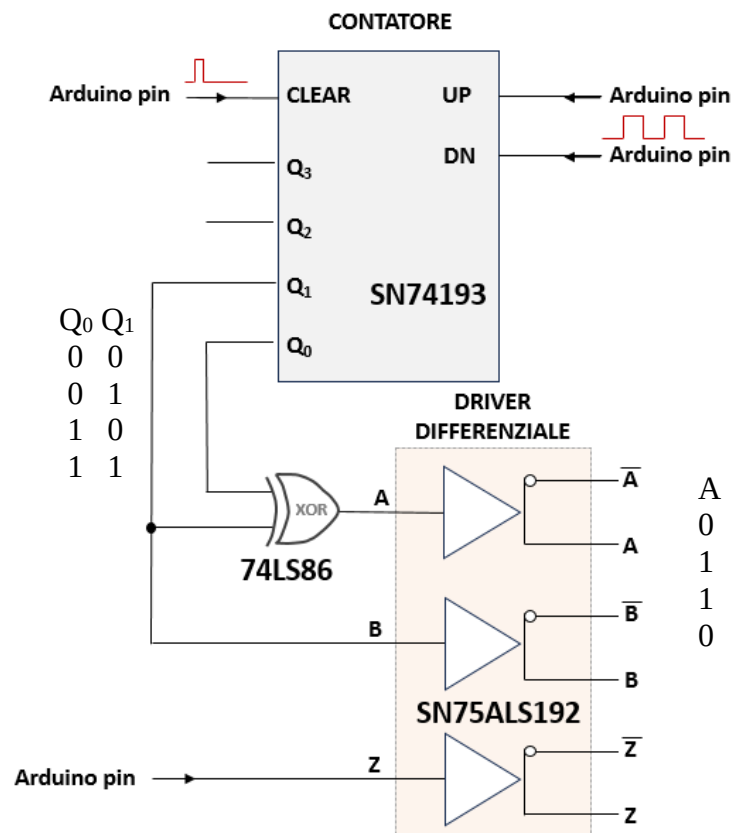


Figura 4: Schema di principio dell'emulatore di encoder

Il microcontrollore governa l'emulatore con soli quattro segnali:

- 1) Clear: impulso per resettare le uscite e partire da una configurazione nota;
- 2) UP: treno di impulsi per contare in avanti $00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$ (rotazione oraria)
- 3) DN: treno di impulsi per contare indietro $00 \rightarrow 10 \rightarrow 11 \rightarrow 01 \rightarrow 00$ (rotazione antioraria)
- 4) "Z": segnale di "zero"

4. Generazione della frequenza base

La generazione di un'onda quadra su un microcontrollore come Arduino Nano si basa sull'utilizzo dei timer hardware, periferiche interne progettate per contare cicli di clock e generare eventi temporizzati con elevata precisione. A differenza delle soluzioni software (come `delay()` o cicli di polling), che introducono errori temporali e carico computazionale, i timer permettono di ottenere segnali stabili e riproducibili, indipendenti dall'esecuzione del codice principale. In particolare, il Timer1 dell'ATmega328P è un contatore a 16 bit che può operare in modalità CTC (Clear Timer on Compare Match). In questa configurazione il timer incrementa il proprio valore fino a raggiungere quello impostato nel registro OCR1A, dopodiché si azzerava automaticamente. Se si abilita anche la modalità di toggle automatico del pin OC1A (D9), il microcontrollore inverte lo stato logico del pin a ogni evento di confronto, generando così un'onda quadra perfettamente simmetrica (duty cycle 50%) senza intervento della CPU. La frequenza del segnale è determinata dalla relazione:

$$f = \frac{f_{CPU}}{2 \cdot prescaler \cdot (OCR1A + 1)}$$

Dove f_{CPU} è tipicamente 16 MHz, mentre il prescaler è un divisore hardware selezionabile (1, 8, 64, 256, 1024) che consente di adattare il range operativo del timer. Poiché il registro OCR1A è a 16 bit (massimo 65535), non è possibile coprire l'intero intervallo 1 Hz – 10 kHz con un solo prescaler: è quindi necessario selezionarlo dinamicamente in funzione della frequenza desiderata. Frequenze elevate richiedono prescaler bassi per mantenere una buona risoluzione temporale, mentre frequenze basse necessitano di prescaler elevati per evitare overflow del contatore. Questo approccio consente di ottenere una generazione di segnale estremamente precisa, con errore limitato alla quantizzazione del timer (tipicamente inferiore allo 0.1%) e senza jitter, rendendolo ideale per applicazioni come clock digitali, modulazione, test di sistemi embedded o simulazione di encoder incrementali.

Dal punto di vista implementativo, la configurazione richiede la manipolazione diretta dei registri del timer, evitando le astrazioni delle librerie Arduino per massimizzare il controllo e le prestazioni.

Il seguente esempio mostra una funzione completa che imposta automaticamente prescaler e valore di confronto per generare un'onda quadra su D9 nel range 1 Hz – 10 kHz:

```
#define PIN_OUT 9 // OC1A: Timer1 hardware output pin

void setupTimer1(uint32_t freq) {

    pinMode(PIN_OUT, OUTPUT); // configure the pin as digital output

    // ===== TIMER RESET =====
    TCCR1A = 0; // clear Timer1 control register A
    TCCR1B = 0; // clear Timer1 control register B

    // ===== HARDWARE OUTPUT =====
    // COM1A0 = 1 → automatic toggle of OC1A pin (D9)
    // the pin changes state at every compare match
    TCCR1A |= (1 << COM1A0);

    // ===== TIMER MODE =====
    // WGM12 = 1 → CTC mode (Clear Timer on Compare Match)
    // the timer counter is reset when it reaches OCR1A
    TCCR1B |= (1 << WGM12);

    uint16_t prescalerBits;
    uint32_t prescaler;

    // ===== PRESCALER SELECTION =====
    // This allows the generator to cover a wide frequency range
    //
    // | CS12 | CS11 | CS10 | Prescaler |
    // | ---- | ---- | ---- | - - - - - |
    // | 0    | 0    | 1    | 1          |
    // | 0    | 1    | 0    | 8          |
    // | 0    | 1    | 1    | 64         |
    // | 1    | 0    | 0    | 256        |
    // | 1    | 0    | 1    | 1024       |
    //
    if (freq >= 2000) {
        prescaler = 1;
        prescalerBits = (1 << CS10); // prescaler = 1
    }
    else if (freq >= 200) {
        prescaler = 8;
        prescalerBits = (1 << CS11); // prescaler = 8
    }
}
```

```

}
else if (freq >= 30) {
prescaler = 64;
prescalerBits = (1 << CS11) | (1 << CS10); // prescaler = 64
}
else if (freq >= 4) {
prescaler = 256;
prescalerBits = (1 << CS12); // prescaler = 256
}
else {
prescaler = 1024;
prescalerBits = (1 << CS12) | (1 << CS10); // prescaler = 1024
}

// ===== OCR1A CALCULATION =====
// Frequency formula:
//
// f = F_CPU / (2 * prescaler * (OCR1A + 1))
//
// solving for OCR1A:
uint32_t ocr = (16000000UL / (2UL * prescaler * freq)) - 1;

OCR1A = ocr; // load compare value into the register

// ===== TIMER START =====
TCCR1B |= prescalerBits; // start the timer with the selected prescaler
}

void setup() {

setupTimer1(1); // example: generate a 1Hz signal

}

void loop() {

// nothing here: the signal is generated entirely in hardware

}

```

Questo codice realizza una generazione completamente hardware del segnale, garantendo massima stabilità e liberando il microcontrollore per altre funzioni (interfaccia utente, comunicazione, controllo).

Un aspetto importante da considerare è che il duty cycle è vincolato al 50% in modalità toggle; per applicazioni che richiedono duty cycle variabile è necessario utilizzare modalità PWM (Fast PWM o Phase Correct). Inoltre, la precisione finale dipende dalla tolleranza del quarzo o dell'oscillatore interno del microcontrollore.

5. Software per Arduino

Abbiamo tutti gli elementi per poter definire le specifiche del software dell'emulatore:

- L'intero sistema è controllato da un microcontrollore della famiglia Arduino, che si occupa della generazione temporizzata dei segnali, della gestione dell'interfaccia utente e dell'aggiornamento del display.
- La frequenza del treno di impulsi generato è regolabile dall'utente tramite pulsanti, che consente di variare il valore della frequenza in modo continuo da 1Hz a 10 kHz.
- Il valore impostato viene mostrato su un display grafico OLED, insieme allo stato logico dell'uscita ZERO.
- La generazione del treno di impulsi non inizi immediatamente all'accensione del dispositivo, ma solo dopo l'esecuzione di una sequenza di reset del contatore e la successiva pressione del pulsante di avvio. Questa scelta permette di sincronizzare correttamente il sistema di test collegato all'emulatore.
- Il funzionamento temporale del sistema è basato sull'utilizzo del Timer1 hardware del microcontrollore, configurato in modalità CTC (Clear Timer on Compare Match). In questa configurazione il timer genera un interrupt periodico quando il contatore raggiunge il valore impostato nel registro di confronto. L'interrupt associato al vettore `TIMER1_COMPA_vect` viene utilizzato per generare il segnale di ZERO in modo sincronizzato con la sequenza degli impulsi e rimane attivo per una durata pari al semiperiodo della frequenza generata, garantendo così la coerenza temporale con il treno di impulsi prodotto.
- Per migliorare la leggibilità dell'interfaccia utente, il software gestisce separatamente la visualizzazione dello stato ZERO sul display rispetto al segnale fisico generato sul pin digitale. In particolare, quando l'uscita ZERO assume livello logico alto, sul display compare la scritta "ON". Tale indicazione rimane visibile per almeno 500 ms, anche nel caso in cui il segnale reale abbia una durata inferiore. Se invece l'uscita ZERO rimane

attiva per un tempo maggiore di 500 ms (ad esempio nel caso di frequenze molto basse), la scritta ON resta visualizzata per tutta la durata effettiva del segnale. Questa logica evita sfarfallii grafici e garantisce che l'utente possa percepire correttamente l'evento anche a frequenze elevate.

Il software che ho preparato, e che trovate nelle ultime pagine, è strutturato secondo una logica modulare in modo da permettere successivi miglioramenti.

6. Schema elettrico

Lo schema elettrico dell'emulatore è relativamente semplice. Il cuore del sistema è rappresentato da Arduino Nano, che gestisce il funzionamento generale del dispositivo e pilota un comune display OLED con interfaccia I²C utilizzato per la visualizzazione dei parametri operativi.

Cinque pulsanti consentono all'utente di accedere a tutte le funzionalità principali del sistema, in particolare:

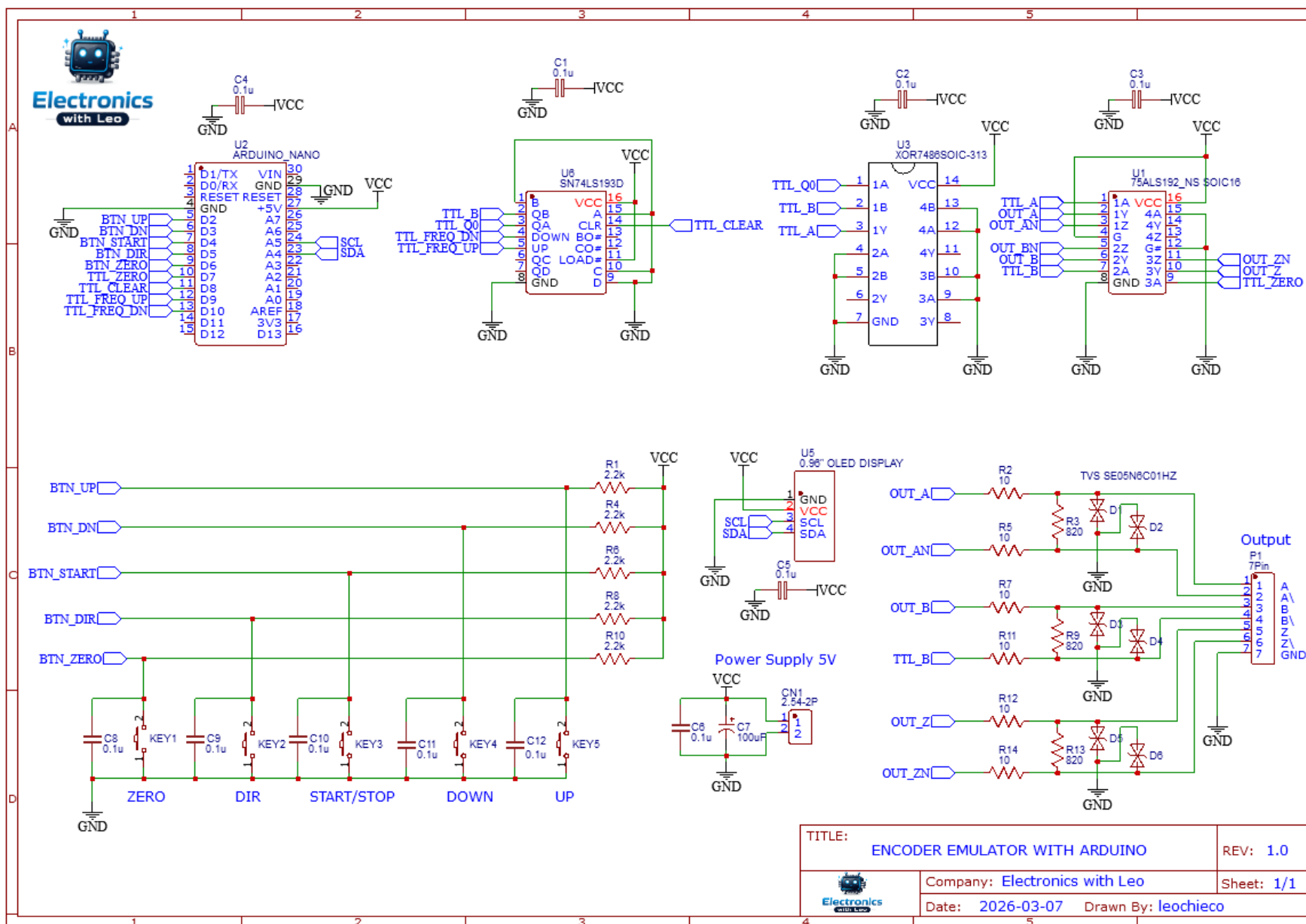
- impostazione della frequenza operativa di base (UP e DOWN);
- avvio e arresto della generazione degli impulsi (START/STOP);
- selezione della direzione della sequenza in quadratura (DIR);
- generazione del segnale di zero (ZERO).

La generazione delle sequenze logiche è affidata a un contatore SN74193, che scandisce ciclicamente i quattro stati necessari al conteggio in quadratura, come descritto in precedenza. Il segnale A viene invece ottenuto tramite una porta XOR contenuta in un 7486, che elabora opportunamente le uscite del contatore.

Successivamente, un 75ALS192 provvede alla conversione dei segnali A, B e Z da configurazione single-ended a differenziale, rendendoli compatibili con le tipiche interfacce utilizzate negli encoder industriali.

Tutte le uscite sono protette mediante diodi TVS, al fine di aumentare la robustezza del sistema nei confronti di sovratensioni o disturbi provenienti dall'ambiente esterno.

Naturalmente si tratta di uno schema di base, che può essere ulteriormente ampliato e migliorato in diversi modi. Ad esempio, sarebbe possibile introdurre un isolamento galvanico tra la CPU (Arduino) e il campo, aumentando l'immunità ai disturbi e la sicurezza del sistema. In questo tutorial, tuttavia, l'obiettivo è semplicemente fornire le basi di partenza per future sperimentazioni e sviluppi.



7. Source code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ===== OLED =====
Adafruit_SSD1306 display(128,64,&Wire,-1);

// ===== PIN =====
#define BTN_UP      2
#define BTN_DOWN    3
#define BTN_START   4
#define BTN_DIR     5
#define BTN_ZERO    6

#define PIN_OC1A    9
#define PIN_OC1B    10

#define PIN_ZERO    7
#define PIN_RESET   8

// ===== PARAMETERS =====
uint32_t frequency = 2;
bool directionForward = true;

const uint16_t debounceTime = 20;

// ===== SYSTEM STATE =====
enum RunState {STOPPED, RESETTING, RUNNING};
RunState runState = STOPPED;

uint32_t resetStartTime = 0;

// ===== ZERO =====
volatile bool triggerZero=false;
volatile uint32_t zeroOffTime=0;

bool zeroOnDisplay=false;
uint32_t zeroDisplayTimer=0;

// ===== BUTTON STRUCT =====
struct Button{
  uint8_t pin;
  bool lastState;
  uint32_t pressTime;
  uint32_t lastRepeat;
  bool shortHandled;
};

Button btnUp    ={BTN_UP,HIGH,0,0,false};
Button btnDown  ={BTN_DOWN,HIGH,0,0,false};
```

```

// ===== DEBOUNCE VAR =====
bool dirState=HIGH;
bool lastDirReading=HIGH;
uint32_t lastDirDebounce=0;

bool startState=HIGH;
bool lastStartReading=HIGH;
uint32_t lastStartDebounce=0;

bool zeroState=HIGH;
bool lastZeroReading=HIGH;
uint32_t lastZeroDebounce=0;

// ===== TIMER =====
void stopTimer()
{
    TCCR1A=0;
    TCCR1B=0;
}

void setupTimer1(uint32_t freq)
{
    TCCR1A=0;
    TCCR1B=0;

    pinMode(PIN_OC1A, OUTPUT);
    pinMode(PIN_OC1B, OUTPUT);

    if(directionForward)
        TCCR1A|=(1<<COM1A0);
    else
        TCCR1A|=(1<<COM1B0);

    TCCR1B|=(1<<WGM12);

    uint32_t prescaler;
    uint16_t bits;

    if(freq>=2000){prescaler=1;bits=(1<<CS10);}
    else if(freq>=200){prescaler=8;bits=(1<<CS11);}
    else if(freq>=30){prescaler=64;bits=(1<<CS11)|(1<<CS10);}
    else if(freq>=4){prescaler=256;bits=(1<<CS12);}
    else {prescaler=1024;bits=(1<<CS12)|(1<<CS10);}

    uint32_t ocr=(16000000UL/(2UL*prescaler*freq))-1;

    OCR1A=ocr;
    OCR1B=ocr;

    TIMSK1=(1<<OCIE1A);

    TCCR1B|=bits;
}

```

```

// ===== TIMER ISR =====
ISR(TIMER1_COMPA_vect)
{
    if(triggerZero)
    {
        digitalWrite(PIN_ZERO, HIGH);

        zeroOffTime=micros()+(500000UL/frequency);

        triggerZero=false;
    }

    if(zeroOffTime && micros()>=zeroOffTime)
    {
        digitalWrite(PIN_ZERO, LOW);
        zeroOffTime=0;
    }
}

// ===== DISPLAY =====
void drawUI()
{
    display.clearDisplay();

    display.drawRect(0, 2, 128, 62, SSD1306_WHITE);

    display.setTextSize(1);

    int16_t x1, y1; uint16_t w, h;
    display.getTextBounds("ENCODER-EMU", 0, 0, &x1, &y1, &w, &h);

    display.fillRect((128-w)/2-2, 0, w+4, h, SSD1306_BLACK);

    display.setCursor((128-w)/2, 0);
    display.print("ENCODER-EMU");

    display.setCursor(4, 18);
    display.print("Freq:");
    display.print(frequency);
    display.print("Hz");

    display.setCursor(4, 32);
    display.print("Dir:");
    display.print(directionForward?"Forward":"Backward");

    display.setCursor(4, 46);
    display.print("State:");
    display.print(runState==RUNNING?"RUN":"STOP");

    display.setCursor(80, 46);
    display.print("Z:");
    display.print(zeroOnDisplay?"ON":"OFF");
}

```

```

    display.display();
}

// ===== STEP =====
uint16_t stepValue(uint32_t t)
{
    if(t<500)return 1;
    else if(t<1500)return 10;
    else if(t<3000)return 100;
    else return 1000;
}

// ===== FREQ BUTTON =====
void handleFreqButton(Button &b,bool increase)
{
    bool state=digitalRead(b.pin);

    if(state==LOW && b.lastState==HIGH)
    {
        b.pressTime=millis();
        b.lastRepeat=millis();
        b.shortHandled=false;
    }

    if(state==LOW)
    {
        uint32_t t=millis()-b.pressTime;

        if(t<500 && !b.shortHandled)
        {
            if(increase)frequency++;
            else if(frequency>1)frequency--;

            b.shortHandled=true;
        }

        else if(t>=500)
        {
            if(millis()-b.lastRepeat>500)
            {
                uint16_t step=stepValue(t);

                if(increase)frequency+=step;
                else if(frequency>step)frequency-=step;

                if(frequency>10000)frequency=10000;

                b.lastRepeat=millis();
            }
        }

        if(runState==RUNNING)

```



```

        setupTimer1(frequency);
    }

    b.lastState=state;
}

// ===== DIR BUTTON =====
void handleDirButton()
{
    bool reading=digitalRead(BTN_DIR);

    if(reading!=lastDirReading)
        lastDirDebounce=millis();

    if(millis()-lastDirDebounce>debounceTime)
    {
        if(reading!=dirState)
        {
            dirState=reading;

            if(dirState==LOW)
            {
                directionForward=!directionForward;

                if(runState==RUNNING)
                    setupTimer1(frequency);
            }
        }
    }

    lastDirReading=reading;
}

// ===== ZERO BUTTON =====
void handleZeroButton()
{
    bool reading=digitalRead(BTN_ZERO);

    if(reading!=lastZeroReading)
        lastZeroDebounce=millis();

    if(millis()-lastZeroDebounce>debounceTime)
    {
        if(reading!=zeroState)
        {
            zeroState=reading;

            if(zeroState==LOW)
            {
                triggerZero=true;

                zeroOnDisplay=true;
                zeroDisplayTimer=millis();
            }
        }
    }
}

```

```

    }
}

lastZeroReading=reading;

if(zeroOnDisplay)
{
    if(!digitalRead(PIN_ZERO) && millis()-zeroDisplayTimer>=500)
        zeroOnDisplay=false;
}
}

// ===== START BUTTON =====
void handleStartButton()
{
    bool reading=digitalRead(BTN_START);

    if(reading!=lastStartReading)
        lastStartDebounce=millis();

    if(millis()-lastStartDebounce>debounceTime)
    {
        if(reading!=startState)
        {
            startState=reading;

            if(startState==LOW)
            {
                if(runState==RUNNING)
                {
                    stopTimer();
                    runState=STOPPED;
                }
                else if(runState==STOPPED)
                {
                    stopTimer();

                    digitalWrite(PIN_RESET,HIGH);

                    resetStartTime=millis();

                    runState=RESETTING;
                }
            }
        }
    }

    lastStartReading=reading;
}

// ===== RESET SEQUENCE =====
void handleResetSequence()

```

```

{
  if(runState==RESETTING)
  {
    if(millis()-resetStartTime>=100)
    {
      digitalWrite(PIN_RESET,LOW);

      setupTimer1(frequency);

      runState=RUNNING;
    }
  }
}

// ===== SETUP =====
void setup()
{
  pinMode(BTN_UP,INPUT_PULLUP);
  pinMode(BTN_DOWN,INPUT_PULLUP);
  pinMode(BTN_START,INPUT_PULLUP);
  pinMode(BTN_DIR,INPUT_PULLUP);
  pinMode(BTN_ZERO,INPUT_PULLUP);

  pinMode(PIN_ZERO,OUTPUT);
  pinMode(PIN_RESET,OUTPUT);

  digitalWrite(PIN_ZERO,LOW);

  display.begin(SSD1306_SWITCHCAPVCC,0x3C);
  display.setTextColor(SSD1306_WHITE);

  digitalWrite(PIN_RESET,HIGH);
  delay(100);
  digitalWrite(PIN_RESET,LOW);

  drawUI();
}

// ===== LOOP =====
void loop()
{
  handleFreqButton(btnUp,true);
  handleFreqButton(btnDown,false);

  handleDirButton();
  handleZeroButton();
  handleStartButton();

  handleResetSequence();

  drawUI();
}

```